UT_3.2_Modificando el documento

1 Trabajar con nodos de texto

Una vez seleccionados uno o varios elementos se pueden modificar: se puede añadir o eliminar texto, añadir o eliminar atributos y añadir o eliminar elementos. **La tabla 1** muestra una descripción general de las principales propiedades y métodos disponibles para este fin, que se tratarán en detalle en las siguientes secciones.

Como se ha mencionado anteriormente, cualquier texto de una página web se representa como nodos de texto dentro del árbol DOM. Ahora veamos cómo se puede acceder al contenido de texto y también modificarlo.

Propiedad/Método	Descripción
`textContent`	Esta propiedad permite acceder al contenido de texto de un nodo.
`nodeValue`	Esta propiedad le permite acceder al contenido de un nodo.
`innerHTML`	Esta propiedad permite acceder al contenido HTML de un nodo.
`createTextNode()`	Este método permite crear nodos de texto.
`createElement()`	Este método permite crear elementos.
`createAttribute()`	Este método permite crear nodos de atributos.
`appendChild()`	Este método permite añadir nodos al árbol DOM.
`removeChild()`	Este método permite eliminar nodos del árbol DOM.

***Tabla 1** Los diferentes métodos y propiedades para modificar elementos*

1.1 Acceso al contenido de texto de un elemento

Para acceder al contenido de texto sin formato de un elemento, lo mejor es utilizar la propiedad `textContent`. Esta propiedad es muy útil porque cualquier marcado HTML dentro del elemento respectivo se ignora y se excluye del valor devuelto. Los dos listados siguientes ilustran esto. En primer lugar, **el listado 1** muestra una lista HTML sencilla con una entrada, en la que el texto contenido se marca utilizando los elementos `<strong>` y `<em>`.

```
<ul id="news">
        <li>
          <strong>Noticias discográficas: </strong>Nuevo álbum de <em>Ben
           Harper</em> a la venta.
```

```
  </li>
</ul>
```

Listado 1 *HTML con elementos anidados*

Si ahora accede a la propiedad `textContent` como se muestra en **el Listado 2**, verá que solo contiene el texto sin formato del elemento `<li>`, pero no las etiquetas `<strong>` y `<em>` que contiene.

```
const textContent = document.querySelector('#news li:nth-
child(1)').textContent;
console.log(textContent);
// Salida: Noticias discográficas: Nuevo álbum de Ben Harper a la venta.
```

Listado 2 *La propiedad textContent ignora el marcado dentro del elemento correspondiente*

Nota

La propiedad `textContent` es muy práctica porque, en la práctica, al acceder al contenido de texto de un elemento, a menudo no es relevante si se ha utilizado marcado adicional ni cuál es este.

1.2 Modificación del contenido de texto de un elemento

Si desea restablecer el contenido de texto de un elemento, también puede utilizar la propiedad `textContent`. Como valor, simplemente pase el nuevo texto, como se muestra en **el Listado 3**. Aquí, se asigna un nuevo texto al elemento de lista utilizado anteriormente.

```
const element = document.querySelector('#news li:nth-child(1)');
element.textContent = 'Noticia de disco: Nuevo álbum de Tool aún no publicado
.';
```

Listado 3 *La propiedad textContent se puede utilizar para restablecer el contenido de texto de un elemento*

Sin embargo, ten en cuenta que `textContent` no te permite añadir marcas HTML: aunque la cadena pasada en el siguiente listado contiene marcas, no se interpretan, sino que se muestran como texto (véase **la figura 1**).

```
const element = document.querySelector('#news li:nth-child(1)');
element.textContent = '<strong>Record news:</strong> New album by
 <em>Tool</em> not yet released.';
```

Listado 4 *El marcado dentro de la cadena especificada no se evalúa*



Figura 1 *El marcado especificado mediante textContent no se evalúa*

textContent frente a innerText

También existe la propiedad innerText, que funciona de manera similar a textContent, pero con algunas diferencias: solo devuelve el texto visible (tiene en cuenta el CSS, por lo que ignora, por ejemplo, los elementos ocultos con display: none) y es más lenta, porque el navegador tiene que calcular los estilos. Durante años no fue estándar y Firefox no la admitía, pero hoy forma parte del estándar HTML y funciona en todos los navegadores. Aun así, recomendamos utilizar textContent salvo que se necesite específicamente el texto tal como se ve en pantalla.

1.3 Modificación del HTML debajo de un elemento

Si desea insertar no solo texto, sino también HTML en un elemento, puede utilizar la propiedad `innerHTML`. Más adelante aprenderá otra posibilidad, la manipulación del DOM, que en la práctica se utiliza a menudo para añadir HTML al DOM. Sin embargo, para empezar, o para añadir componentes HTML sencillos, `innerHTML` es suficiente. En **el Listado 5** se muestra un ejemplo**:** se añade el mismo bloque HTML que en **el Listado 4,** pero esta vez también se interpreta como HTML (véase **la Figura 2)**.

```
const element = document.querySelector('#news li:nth-child(1)');
element.innerHTML = '<strong>Record news:</strong> New album by <em>Tool
 </em> not yet released.'
```

Listado 5 *La propiedad innerHTML se utiliza para evaluar el marcado contenido en la cadena pasada*

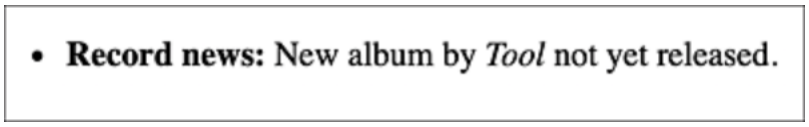


Figura 2 *Como era de esperar, se evalúa el HTML insertado mediante innerHTML*

Por el contrario, también se puede utilizar `innerHTML` para leer el contenido HTML de un elemento. Al igual que con `textContent`, el resultado es una cadena que ahora contiene no solo el contenido del texto, sino también el marcado HTML (véase **el Listado 6**).

```
const innerHTML = document.querySelector('#news li:nth-child(1)').innerHTML;
console.log(innerHTML);
// Salida: <strong>Noticias discográficas: </strong>Nuevo álbum de
// <em>Ben Harper</em> a la venta.
```

Listado 6 *La propiedad innerHTML también contiene el marcado HTML*

1.4 Creación y adición de nodos de texto

Como alternativa a las opciones mostradas para acceder o modificar el texto dentro de una página web a través de las propiedades `textContent` e `innerHTML`, existe la posibilidad de crear nodos de texto y añadirlos manualmente al árbol DOM. Para ello, la

API DOM proporciona el método `createTextNode()`. En **el Listado 7,** se crea un nodo de texto (con el texto `Ejemplo`) mediante este método y, a continuación, se añade a un elemento existente como nodo secundario utilizando el método `appendChild()` (más información al respecto más adelante). (Este segundo paso es necesario porque el nodo de texto aún no se ha añadido al árbol DOM mediante el método `createTextNode()`).

```
const element = document.getElementById('container');
const textNode = document.createTextNode('Example');
element.appendChild(textNode);
```

***Listado** 7 Creación y adición de un nodo de texto*

Otros métodos para crear nodos

Además del método `createTextNode()`, existen otros métodos para crear nodos, entre ellos `createElement()` para crear nodos de elemento (Sección 2) y `createAttribute()` para crear nodos de atributo (**Sección 3.3**).

Métodos de nodos de documento

El método `createTextNode()` y también los métodos `createElement()` y `createAttribute()` descritos más adelante solo están disponibles en el nodo de documento (es decir, el objeto `document`). Estos métodos no se pueden invocar en otros nodos (y, por lo tanto, tampoco en elementos).

2 Trabajar con elementos

Los elementos también se pueden crear manualmente mediante métodos y, a continuación, añadirse al árbol DOM (a diferencia de lo que ocurre con la propiedad `innerHTML`, en la que se pasan los elementos HTML de forma indirecta en forma de texto que se asigna a la propiedad).

En esta sección, le mostraremos cómo crear y añadir elementos utilizando estos métodos y cómo trabajar con elementos en general.

Para crear nodos DOM, hay dos métodos:

```
document.createElement(tag)
```

Crea un nuevo nodo elemento con la etiqueta HTML dada:

```
let div = document.createElement('div');
document.createTextNode(text)
```

Crea un nuevo nodo texto con el texto dado:

```
let textNode = document.createTextNode('Aquí estoy');
```

La mayor parte del tiempo necesitamos crear nodos de elemento, como el div para el mensaje.

Creando el mensaje

Crear el div de mensaje toma 3 pasos:

```
// 1. Crear elemento <div>
let div = document.createElement('div');

// 2. Establecer su clase a "alert"
div.className = "alert";

// 3. Agregar el contenido
div.innerHTML = "<strong>¡Hola!</strong> Usted ha leído un importante mensaje.";
```

Hemos creado el elemento. Pero hasta ahora solamente está en una variable llamada div, no aún en la página, y no la podemos ver.

Métodos de inserción

Para hacer que el `div` aparezca, necesitamos insertarlo en algún lado dentro de `document`. Por ejemplo, en el elemento `<body>`, referenciado por `document.body`.

Hay un método especial `append` para ello: `document.body.append(div)`.

El código completo:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  let div = document.createElement('div');
  div.className = "alert";
  div.innerHTML = "<strong>¡Hola!</strong> Usted ha leído un importante mensaje.";
```

```
  document.body.append(div);
</script>
```

Aquí usamos el método `append` sobre `document.body`, pero podemos llamar `append` sobre cualquier elemento para poner otro elemento dentro de él. Por ejemplo, podemos añadir algo a `<div>` llamando `div.append(anotherElement)`.

Aquí hay más métodos de inserción, ellos especifican diferentes lugares donde insertar:

`node.append(...nodos o strings)` – agrega nodos o strings *al final* de `node`,
`node.prepend(...nodos o strings)` – inserta nodos o strings *al principio* de `node`,
`node.before(...nodos o strings)` – inserta nodos o strings *antes* de `node`,
`node.after(...nodos o strings)` – inserta nodos o strings *después* de `node`,
`node.replaceWith(...nodos o strings)` – reemplaza `node` con los nodos o strings dados.

Los argumentos de estos métodos son una lista arbitraria de lo que se va a insertar: nodos DOM o strings de texto (estos se vuelven nodos de texto automáticamente).

Veámoslo en acción.

Aquí tenemos un ejemplo del uso de estos métodos para agregar items a una lista y el texto antes/después de él. Fíjate en que primero seleccionamos la lista con getElementById(): aunque el navegador crea automáticamente variables globales con el id de cada elemento (ol, div...), no conviene depender de ellas.

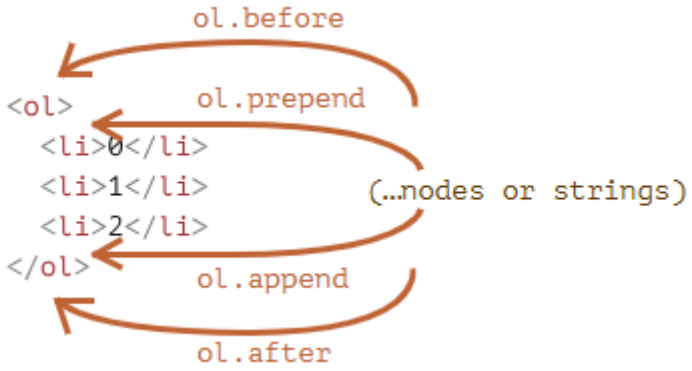
```
<ol id="ol">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>

<script>
  const ol = document.getElementById('ol');
  ol.before('before'); // inserta el string "before" antes de <ol>
  ol.after('after'); // inserta el string "after" después de <ol>

  let liFirst = document.createElement('li');
  liFirst.innerHTML = 'prepend';
  ol.prepend(liFirst); // inserta liFirst al principio de <ol>

  let liLast = document.createElement('li');
  liLast.innerHTML = 'append';
  ol.append(liLast); // inserta liLast al final de <ol>
</script>
```

Aquí la representación visual de lo que hacen los métodos:



Entonces la lista final será:

```
before
<ol id="ol">
  <li>prepend</li>
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>append</li>
</ol>
after
```

Como dijimos antes, estos métodos pueden insertar múltiples nodos y piezas de texto en un simple llamado.

Por ejemplo, aquí se insertan un string y un elemento:

```
<div id="div"></div>
<script>
  const div = document.getElementById('div');
  div.before('<p>Hola</p>', document.createElement('hr'));
</script>
```

Nota que el texto es insertado “como texto” y no “como HTML”, escapando apropiadamente los caracteres como <, >.

Entonces el HTML final es:

```
&lt;p&gt;Hola&lt;/p&gt;
<hr>
<div id="div"></div>
```

En otras palabras, los strings son insertados en una manera segura, tal como lo hace `elem.textContent`.

Entonces, estos métodos solo pueden usarse para insertar nodos DOM como piezas de texto.

Pero ¿y si queremos insertar un string HTML "como html", con todas las etiquetas y demás funcionando, de la misma manera que lo hace `elem.innerHTML`?

insertAdjacentHTML/Text/Element

Para ello podemos usar otro método, muy versátil: `elem.insertAdjacentHTML(where, html)`.

El primer parámetro es una palabra código que especifica dónde insertar relativo a `elem`. Debe ser uno de los siguientes:

`"beforebegin"` – inserta `html` inmediatamente antes de `elem`
`"afterbegin"` – inserta `html` en `elem`, al principio
`"beforeend"` – inserta `html` en `elem`, al final
`"afterend"` – inserta `html` inmediatamente después de `elem`

El segundo parámetro es un string HTML, que es insertado "como HTML".

Por ejemplo:

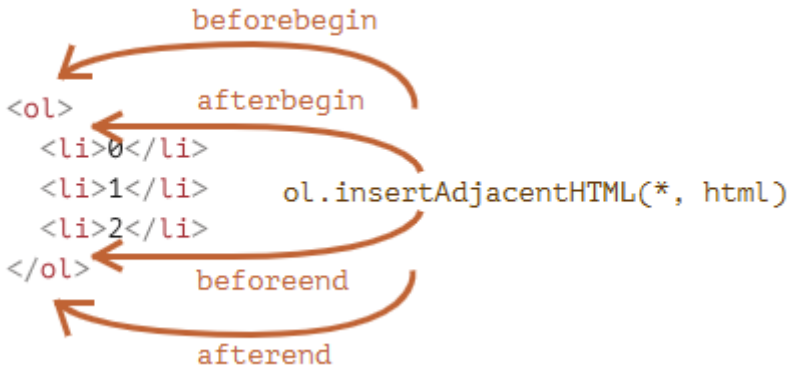
```
<div id="div"></div>
<script>
  const div = document.getElementById('div');
  div.insertAdjacentHTML('beforebegin', '<p>Hola</p>');
  div.insertAdjacentHTML('afterend', '<p>Adiós</p>');
</script>
```

...resulta en:

```
<p>Hola</p>
<div id="div"></div>
<p>Adiós</p>
```

Así es como podemos añadir HTML arbitrario a la página.

Aquí abajo, la imagen de las variantes de inserción:



Fácilmente podemos notar similitudes entre esta imagen y la anterior. Los puntos de inserción son los mismos, pero este método inserta HTML.

El método tiene dos hermanos:

`elem.insertAdjacentText(where, text)` – la misma sintaxis, pero un string de `texto` es insertado "como texto" en vez de HTML,
`elem.insertAdjacentElement(where, elem)` – la misma sintaxis, pero inserta un elemento.

Ellos existen principalmente para hacer la sintaxis "uniforme". En la práctica, solo `insertAdjacentHTML` es usado la mayor parte del tiempo. Porque para elementos y texto, tenemos los métodos `append/prepend/before/after`: son más cortos para escribir y pueden insertar piezas de texto y nodos.

Entonces tenemos una alternativa para mostrar un mensaje:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  document.body.insertAdjacentHTML("afterbegin", `<div class="alert">
    <strong>¡Hola!</strong> Usted ha leído un importante mensaje.
  </div>`);
</script>
```

Clonando nodos: cloneNode

¿Cómo insertar un mensaje similar más?

Podríamos hacer una función y poner el código allí. Pero la alternativa es *clonar* el `div` existente, y modificar el texto dentro si es necesario.

A veces, cuando tenemos un elemento grande, esto es más simple y rápido.

La llamada `elem.cloneNode(true)` crea una clonación "profunda" del elemento, con todos los atributos y subelementos. Si llamamos `elem.cloneNode(false)`, la clonación se hace sin sus elementos hijos.

Un ejemplo de copia del mensaje:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>
<div class="alert" id="div">
  <strong>¡Hola!</strong> Usted ha leído un importante mensaje.
</div>
<script>
  const div = document.getElementById('div');
  let div2 = div.cloneNode(true); // clona el mensaje
  div2.querySelector('strong').innerHTML = '¡Adiós!'; // altera el clon

  div.after(div2); // muestra el clon después del div existente
</script>
```

DocumentFragment

`DocumentFragment` es un nodo DOM especial que sirve como contenedor para trasladar listas de nodos.

Podemos agregarle nodos, pero cuando lo insertamos en algún lugar, lo que se inserta es su contenido.

Por ejemplo, `getListContent` de abajo genera un fragmento con items `<li>`, que luego son insertados en `<ul>`:

```
<ul id="ul"></ul>

<script>
function getListContent() {
  let fragment = new DocumentFragment();

  for(let i=1; i<=3; i++) {
    let li = document.createElement('li');
    li.append(i);
    fragment.append(li);
  }
  return fragment;
}

const ul = document.getElementById('ul');
ul.append(getListContent()); // (*)
</script>
```

Nota que a la última línea `(*)` añadimos `DocumentFragment`, pero este despliega su contenido. Entonces la estructura resultante será:

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
</ul>
```

Es raro que `DocumentFragment` se use explícitamente. ¿Por qué añadir un tipo especial de nodo si en su lugar podemos devolver un array de nodos? El ejemplo reescrito:

```
<ul id="ul"></ul>

<script>
function getListContent() {
  let result = [];

  for(let i=1; i<=3; i++) {
    let li = document.createElement('li');
    li.append(i);
    result.push(li);
  }

  return result;
}
```

```
const ul = document.getElementById('ul');
ul.append(...getListContent()); // append +  el operador "..." = ¡amigos!
</script>
```

2.2 Eliminación de elementos y nodos

Para eliminar elementos (o, más en general, nodos) de un elemento padre (o, más en general, un nodo padre), se puede utilizar el método `removeChild()`. Este método espera que se elimine el elemento (o nodo) y también lo devuelve como valor de retorno. En **el Listado 10** (basado en los listados de la sección anterior), se puede ver un método para ordenar los datos de tabla (`sortByFirstName()`) que aprovecha el método `removeChild()` para eliminar todos los nodos hijos y elementos hijos del cuerpo de la tabla (es decir, todas las filas de la tabla).

```
function sortByFirstName() {
  const tableBody = document.querySelector('#contact-table tbody');
  while (tableBody.firstChild !== null) {
    tableBody.removeChild(tableBody.firstChild);
  }
  contacts.sort(function(contact1, contact2) {
    return contact1.firstName.localeCompare(contact2.firstName);
  })
  createTable();
}
```

***Listado 10** Ejemplo de uso del método removeChild()*

2.3 Los diferentes tipos de elementos HTML

Cada elemento HTML está representado por un tipo de objeto específico dentro de un árbol DOM. Estos se definen en una extensión de la API DOM denominada especificación DOM HTML. Por ejemplo, los enlaces (elementos `<a>`) están representados por el tipo `HTMLAnchorElement`, las tablas (elementos `<table>`) por el tipo `HTMLTableElement`, etc. En **la tabla 2** se ofrece una descripción general de los diferentes elementos HTML y sus tipos de objetos correspondientes**.** Los tipos obsoletos aparecen en cursiva en la tabla.

El supertipo HTMLElement

Todos los tipos de objetos tienen el mismo supertipo, es decir, el tipo `HTMLElement`. A los elementos que no tienen un tipo específico se les asigna el tipo `HTMLElement` de forma predeterminada.

Elementos obsoletos

En aras de la exhaustividad, los elementos y tipos de objetos que han quedado obsoletos se siguen incluyendo en la tabla en cursiva.

Elemento HTML	Tipo
`<a>`	`HTMLAnchorElement`
`<abbr>`	`HTMLElement`
`<acronym>`	`HTMLElement`
`<address>`	`HTMLElement`
`<applet>`	*`HTMLAppletElement`*
`<area>`	`HTMLAreaElement`
`<audio>`	`HTMLAudioElement`
`<b>`	`HTMLElement`
`<base>`	`HTMLBaseElement`
`<basefont>`	*`HTMLBaseFontElement`*
`<bdo>`	`HTMLElement`
`<big>`	`HTMLElement`
`<blockquote>`	`HTMLQuoteElement`
`<body>`	`HTMLBodyElement`
` `	`HTMLBRElement`
`<button>`	`HTMLButtonElement`
`<caption>`	`HTMLTableCaptionElement`
`<canvas>`	`HTMLCanvasElement`
`<center>`	`HTMLElement`
`<cite>`	`HTMLElement`
`<code>`	`HTMLElement`
`<col>, <colgroup>`	`HTMLTableColElement`
`<data>`	`HTMLDataElement`
`<datalist>`	`HTMLDataListElement`
`<dd>`	`HTMLElement`
`<del>`	`HTMLModElement`
`<dfn>`	`HTMLElement`
`<dir>`	*`HTMLDirectoryElement`*
`<div>`	`HTMLDivElement`
`<dl>`	`HTMLDListElement`
`<dt>`	`HTMLElement`

	HTMLElement
<embed>	HTMLEmbedElement
<fieldset>	HTMLFieldSetElement
<font>	*HTMLFontElement*
<form>	HTMLFormElement
<frame>	*HTMLFrameElement*
<frameset>	*HTMLFrameSetElement*
<h1>, <h2>, <h3>, <h4>, <h5>, <h6>	HTMLHeadingElement
<head>	HTMLHeadElement
<hr>	HTMLHRElement
<html>	HTMLHtmlElement
<i>	HTMLElement
<iframe>	HTMLIFrameElement
<img>	HTMLImageElement
<input>	HTMLInputElement
<ins>	HTMLModElement
<isindex>	*HTMLIsIndexElement*
<kbd>	HTMLElement
<keygen>	HTMLKeygenElement
<label>	HTMLLabelElement
<legend>	HTMLLegendElement
	HTMLLIElement
<link>	HTMLLinkElement
<map>	HTMLMapElement
<audio>, <video> (tipo base)	HTMLMediaElement
<menu>	*HTMLMenuElement*
<meta>	HTMLMetaElement
<meter>	HTMLMeterElement
<noframes>	HTMLElement
<noscript>	HTMLElement
<object>	HTMLObjectElement
	HTMLOListElement
<optgroup>	HTMLOptGroupElement
<option>	HTMLOptionElement
<output>	HTMLOutputElement

<p>	HTMLParagraphElement
<param>	HTMLParamElement
<pre>	HTMLPreElement
<progress>	HTMLProgressElement
<q>	HTMLQuoteElement
<s>	HTMLElement
<samp>	HTMLElement
<script>	HTMLScriptElement
<select>	HTMLSelectElement
<small>	HTMLElement
<source>	HTMLSourceElement
	HTMLSpanElement
<strike>	HTMLElement
	HTMLElement
<style>	HTMLStyleElement
<sub>	HTMLElement
<sup>	HTMLElement
<table>	HTMLTableElement
<tbody>	HTMLTableSectionElement
<td>	HTMLTableCellElement
<textarea>	HTMLTextAreaElement
<tfoot>	HTMLTableSectionElement
<th>	HTMLTableCellElement
<thead>	HTMLTableSectionElement
<time>	HTMLTimeElement
<title>	HTMLTitleElement
<tr>	HTMLTableRowElement
<tt>	HTMLElement
<u>	HTMLElement
<track>	HTMLTrackElement
	HTMLUListElement
<var>	HTMLElement
	HTMLUnknownElement
<video>	HTMLVideoElement

***Tabla 2** Los diferentes elementos HTML y sus respectivos tipos en JavaScript*

El Listado 11 muestra el principio de estos tipos de objetos utilizando el ejemplo de las tablas representadas por el tipo `HTMLTableElement`. Este tipo, al igual que todos los demás tipos que se muestran en **la Tabla 2**, tiene propiedades individuales correspondientes al tipo, incluida la propiedad `caption`, que contiene el subtítulo de una tabla (y es `nula` en el ejemplo, porque la tabla de contactos de la UT 3.1 no tiene un elemento <caption>); la propiedad `tHead`, que contiene el área de encabezado, es decir, el elemento `<thead>`, de una tabla; la propiedad `tBodies`, que contiene los distintos cuerpos de la tabla, es decir, los elementos `<tbody>`, de una tabla; la propiedad `rows`, que contiene las filas de la tabla (incluidas las del área de encabezado); y la propiedad `tFoot`, que contiene el área de pie de página, es decir, el elemento `<tfoot>`.

```
const table = document.querySelector('table');
console.log(Object.getPrototypeOf(table)); // HTMLTableElement


console.log(table.caption);                // null
console.log(table.tHead);                  // thead
console.log(table.tBodies);                // [tbody]
console.log(table.rows);                   // [tr, tr, tr, tr, tr]
console.log(table.tFoot);                  // null
```

***Listado 11** Cada elemento HTML está representado por su propio tipo de objeto*

Además de las propiedades individuales, los diferentes tipos de objetos también tienen diferentes métodos: por ejemplo, el tipo `HTMLTableElement`, que se muestra en **el Listado 12**, contiene el método `insertRow()` (entre otros), que se puede utilizar para crear directamente una nueva fila de tabla (sin crear manualmente los elementos HTML correspondientes mediante `document.createElement()`). Este método devuelve un objeto del tipo `HTMLTableRowElement`, que, a su vez, comprende el método `insertCell()` (entre otros), mediante el cual se puede añadir directamente una nueva celda de tabla a la fila correspondiente. En el ejemplo, se crea de esta manera una nueva fila de tabla con tres celdas. Esto es mucho más claro que crear cada fila y cada celda con document.createElement(), ¿no crees?

```
const newRow   = table.insertRow(1);
const newCellFirstName  = newRow.insertCell(0);
newCellFirstName.textContent = 'Bob';
const newCellLastName  = newRow.insertCell(1);
newCellLastName.textContent = 'Doe';
const newCellEmail  = newRow.insertCell(2);
newCellEmail.textContent = 'bob.doe@javascripthandbuch.de ';
```

***Listado 12** Los diferentes tipos de objetos también tienen métodos individuales*

Nombres de propiedades frente a nombres de elementos

Tenga en cuenta que las propiedades de los objetos como `tHead`, `tBodies` y `tFoot` se escriben en notación CamelCase, mientras que los elementos HTML correspondientes se escriben en minúsculas (`<thead>`, `<tbody>`, `<tfoot>`).

3 Trabajar con atributos

La API DOM proporciona varios métodos para trabajar con atributos:

Puede utilizar el método `getAttribute()` para acceder a los atributos de un elemento (**Sección 3.1**).

Puede utilizar el método `setAttribute()` para cambiar el valor de un atributo o para añadir atributos a un elemento (**Sección 3.2**).

Puede crear nodos de atributos mediante el método `createAttribute()` y añadirlos utilizando `setAttributeNode()` (**Sección 3.3**).

Puede eliminar atributos utilizando el método `removeAttribute()` (**Sección 3.4**).

3.1 Lectura del valor de un atributo

Para acceder al valor de un atributo, utilice el método `getAttribute()` en el elemento correspondiente. Como parámetro, el método espera el nombre del atributo HTML correspondiente. A continuación, devuelve el valor del atributo correspondiente. Utilice el código HTML del Listado 13 como punto de partida: muestra un enlace (un elemento `<a>`) con los atributos `id`, `class` y `href`.

```
<a id="home" class="link" href="index.html">Inicio</a>
```

***Listado 13** Un enlace HTML*

Para acceder a estos atributos, utilice el método `getAttribute()`, como se muestra en **el Listado 14**

```
const element = document.getElementById('home');
console.log(element.getAttribute('id'));    // home
console.log(element.getAttribute('class')); // link
console.log(element.getAttribute('href')); // index.html
```

***Listado 14** Puede acceder a los atributos de un elemento HTML utilizando el método getAttribute()*

Los atributos de un elemento suelen estar también disponibles como propiedades con el mismo nombre, aunque el atributo `class` es una excepción: se puede acceder a este atributo a través de la propiedad `className`. **El Listado 15** muestra un ejemplo correspondiente: los atributos `id` y `href` se pueden leer utilizando las propiedades del mismo nombre, y el atributo `class` a través de la propiedad `className`.

```
console.log(element.id);        // home
console.log(element.className); // link
console.log(element.href);      // http://…/index.html (URL absoluta)
```

Listado 15 *Los atributos de un elemento también están disponibles como propiedades*

Sin embargo, debe tener en cuenta lo siguiente: para dos atributos, el acceso mediante el método `getAttribute()` devuelve un valor diferente al del acceso directo mediante la propiedad. Para el atributo `style`, el método `getAttribute()` solo devuelve el texto contenido como valor en el atributo. Por otro lado, el acceso a la propiedad `style` devuelve un objeto del tipo `CSSStyleDeclaration`, que se puede utilizar para acceder a la información CSS correspondiente en detalle. Además, todos los atributos que se pueden utilizar para definir controladores de eventos devuelven el código JavaScript que se va a ejecutar como un objeto de función a través de la propiedad correspondiente (por ejemplo, `onclick`). Si, por el contrario, se accede al atributo correspondiente mediante el método `getAttribute()`, se devuelve el nombre de la función que se va a ejecutar como texto.

Eche un vistazo al **Listado 16** y **al Listado 17**. El primero muestra un botón HTML con varios atributos, incluidos un atributo `de estilo` y un atributo `onclick`. En el segundo, puede ver cómo se accede a ambos a través de la propiedad del mismo nombre y a través del método `getAttribute()`, respectivamente.

```
<button id="create" class="link" style="background-color: green"
onclick="createContact()">Crear contacto</button>
```

Listado 16 *Un botón HTML*

```
const button = document.getElementById('create');
console.log(button.onclick);                        // Salida del
                                                   // objeto de función
console.log(typeof button.onclick);                 // Salida: function
console.log(button.getAttribute('onclick'));       // createContact()
console.log(typeof button.getAttribute('onclick')); // Salida: string
console.log(button.style);                          // Salida de
                                                   // CSSStyleDeclaration
console.log(typeof button.style);                   // Salida: object
console.log(button.getAttribute('style'));         // background-color: green
console.log(typeof button.getAttribute('style'));  // Salida: string
```

Listado 17 *El acceso a los controladores de eventos y al atributo style devuelve diferentes valores de retorno según el tipo de acceso*

La razón por la que el acceso directo a los atributos del controlador de eventos, como `onclick`, no devuelve una cadena sino una función es que esta propiedad le permite

definir controladores de eventos para el elemento respectivo. Es decir, puede asignar objetos de función a esta propiedad.

La razón por la que el acceso directo al atributo `style` no devuelve una cadena es que este atributo se puede utilizar para acceder mediante programación a la información CSS del elemento respectivo, incluso con acceso de escritura (como ya se ha visto en este tema).

3.2 Cambiar el valor de un atributo o añadir un nuevo atributo

Para cambiar el valor de un atributo o añadir un nuevo atributo, utilice el método `setAttribute()` en el elemento cuyo atributo desea cambiar. Este método requiere dos parámetros: el nombre del atributo y el nuevo valor. Si el elemento correspondiente ya tiene un atributo con el mismo nombre, el valor de ese atributo se sobrescribirá con el nuevo valor. Si el atributo aún no existe, se añadirá un nuevo atributo correspondiente al elemento. En **el Listado 18** se muestra un ejemplo**.** Este listado cambia las propiedades `class, href` y `target` del elemento de enlace seleccionado anteriormente.

```
const element = document.getElementById('home');
element.setAttribute('class', 'link active');
element.setAttribute('href', 'newlink.html');
element.setAttribute('target', '_blank');
console.log(element.getAttribute('class'));  // link active
console.log(element.getAttribute('href'));   // newlink.html
console.log(element.getAttribute('target')); // _blank
```

Listado 18 *Puede cambiar los atributos existentes de un elemento HTML o añadir nuevos atributos utilizando el método setAttribute().*

Alternativamente, también puede cambiar los valores de los atributos o añadir nuevos atributos a través de las propiedades del objeto (normalmente) del mismo nombre (véase el Listado 19).

```
element.className = 'link active highlighted';
element.href = 'anotherLink.html';
element.target = '_self';
console.log(element.getAttribute('class'));  // link active highlighted
console.log(element.getAttribute('href'));   // anotherLink.html
console.log(element.getAttribute('target')); // _self
```

Listado 19 *Los atributos también se pueden cambiar directamente a través de las propiedades correspondientes*

Nota

En segundo plano, cuando se utiliza el método `setAttribute()`, se crea (o se actualiza) un nodo de atributo asociado al elemento correspondiente. Ojo: los atributos no son nodos hijos del elemento; no aparecen en childNodes, sino en la propiedad attributes.

3.3 Creación y adición de nodos de atributos

Al igual que con los textos normales y cuando se trabaja con elementos, también se pueden crear atributos como nodos de atributo utilizando un método especial, el método `createAttribute()`. Como argumento, este método espera el nombre del atributo que se va a crear. A continuación, devuelve el nuevo nodo de atributo. Al igual que los nodos de texto y los nodos de elemento mencionados anteriormente, este nodo tampoco se integra directamente en el árbol DOM. Debe hacerlo manualmente utilizando el método `setAttributeNode()` para el elemento correspondiente (véase **el Listado 20)**.

```
const element = document.getElementById('home');
const attribute = document.createAttribute('target');
attribute.value = '_blank';
element.setAttributeNode(attribute);
console.log(element.getAttribute('target')); // _blank
```

Listado 20 *Creación y adición de un nodo de atributo*

3.4 Eliminación de atributos

Puede eliminar atributos de un elemento utilizando el método `removeAttribute()`. En **el Listado 21**, se utiliza para eliminar los dos atributos `class` y `href` del elemento link. Posteriormente, los dos atributos devuelven el valor `null`.

```
const element = document.getElementById('home');
element.removeAttribute('class');
element.removeAttribute('href');
console.log(element.getAttribute('class'));  // null
console.log(element.getAttribute('href'));   // null
```

Listado 21 *Se pueden eliminar atributos de un elemento HTML utilizando el método removeAttribute()*

3.5 Acceso a clases CSS

Aunque ya lo has visto (al menos parcialmente) en algunos ejemplos a lo largo del tema, explicaremos brevemente cómo leer las clases CSS de un elemento.

En primer lugar, está la conocida propiedad `className`, que está disponible en todos los elementos de una página web (es decir, en todos los nodos de elementos). Esta propiedad simplemente contiene el valor del atributo `class` del elemento correspondiente como una cadena. Si el elemento tiene varias clases CSS, estos nombres de clase se separan por espacios dentro de la cadena.

En el pasado, esto a veces daba lugar a código engorroso cuando, por ejemplo, se quería añadir nuevas clases CSS a un elemento o, lo que es peor, eliminar clases CSS existentes. ¿Por qué engorroso? Porque en todos y cada uno de los casos había que analizar el valor del atributo.

Esto se tuvo en cuenta en la versión 4 de la API DOM. Desde entonces, los elementos (en la API DOM, no en HTML) tienen la propiedad adicional `classList`, que contiene las clases CSS como una lista. Añadir y eliminar clases CSS individuales a un elemento o de un elemento se ha vuelto mucho más fácil desde entonces:

- El método `add()` se puede utilizar para añadir nuevas clases CSS a la lista.
- El método `remove()` permite eliminar clases CSS de la lista.
- Mediante el método `toggle()`, se pueden alternar las clases CSS. Es decir, si la clase CSS existe en la lista, se elimina; si no existe, se añade. Este método puede incluso vincularse a condiciones booleanas.
- Además, el método `contains()` se puede utilizar para comprobar si una clase CSS está contenida en la lista.

El listado 22 muestra algunos ejemplos de estos métodos.

```
const element = document.getElementById('home');
console.log(element.classList);                    // ["link"]
element.classList.add('active');                  // Añadir clase
console.log(element.classList);                    // ["link", "active"]
element.classList.remove('active');               // Eliminar clase
console.log(element.classList);                    // ["link"]
element.classList.toggle('active');                // Alternar clase
console.log(element.classList);                    // ["link", "active"]
element.classList.toggle('active');               // Alternar clase
console.log(element.classList);                    // ["link"]
console.log(element.classList.contains('link'));   // true
console.log(element.classList.contains('active')); // false
const i = 5;
const condition = i > 0;
element.classList.toggle('active', condition);     // Alternar clase
console.log(element.classList);                    // ["link", "active"]
```

Listado 22 *La propiedad classList de los elementos facilita mucho el trabajo con clases CSS*

4 Resumen

En este tema, has aprendido a acceder y modificar dinámicamente el contenido de una página web utilizando JavaScript. Resumamos los puntos principales:

- *El modelo de objetos de documento* representa el modelo de una página web, una estructura jerárquica en forma de árbol.
- Los componentes individuales de esta estructura de árbol se denominan *nodos*, y hay diferentes tipos de nodos. Los más importantes son *los nodos de documento*, *los nodos de elemento*, *los nodos de texto* y *los nodos de atributo*. Además, los nodos de elemento se representan mediante diferentes tipos, empezando por el tipo `HTMLElement`.

- *La API DOM* define propiedades y métodos que te permiten acceder o modificar los datos de una página web.
- Por ejemplo, puede añadir elementos, eliminar elementos, modificar textos y añadir o eliminar atributos utilizando la API DOM.
- Los elementos de una página web se pueden seleccionar de varias maneras: por ID, por clase CSS, por nombre de elemento, por atributo `de nombre` y por selector CSS.
- A partir de un elemento o nodo, se pueden seleccionar el elemento padre/nodo padre, los elementos hijos/nodos hijos y los elementos hermanos/nodos hermanos mediante diversas propiedades.
- La propiedad `textContent` se puede utilizar para acceder o establecer el contenido de texto de un nodo, mientras que la propiedad `innerHTML` se puede utilizar para acceder al contenido HTML de un elemento.
- Puedes crear nodos de texto mediante `createTextNode()`, nodos de elemento mediante `createElement()` y nodos de atributo mediante `createAttribute()`.
- Después de crear un nodo, primero debe añadirlo al árbol DOM utilizando diferentes métodos: `insertBefore()`, `appendChild()` y `replaceChild()`. También existen sus versiones modernas, más cómodas: append(), prepend(), before(), after() y replaceWith(). Para eliminarlo se usa removeChild() o, simplemente, remove().